

Using CACE for a Chipalooza CMOS5L Schematic-Review Submission

An excerpt-and-walkthrough guide, adapted from the JKU *Analog
(Integrated) Circuit Design* course

Compiled for Christoph Maier

22 August 2026

Contents

Contents	2
1. What CACE Is, and Why It Matters for Schematic Review	2
2. CACE Project Layout	3
3. Anatomy of the YAML Datasheet	4
3.1 Metadata and authorship	4
3.2 Paths	4
3.3 Pins	4
3.4 Default conditions	5
3.5 Parameters — spec, tool, conditions, plot	6
4. Testbench Templates and Token Substitution	7
5. Running CACE and Reading the Report	8
6. PVT Corner Sweeps	8
7. Monte Carlo and Mismatch Analysis	9
8. Post-Layout Characterization	10
9. Per-PDK Adaptation: SG13G2/CMOS5L, GF180MCU, SKY130	11
9.1 IHP SG13G2 (full stack) and SG13CMOS5L (restricted) — primary, immediate	11
9.2 GF180MCU — second priority, time-critical	12
9.3 SKY130 — third priority, lower urgency	12
9.4 Summary table	13
10. Model-Accuracy Caveats	13
11. Mapping CACE Output to Chipalooza Schematic-Review Expectations	14
12. Appendix: Command Cheat-Sheet and Blank Datasheet Skeleton	14
12.1 Commands	14
12.2 Blank datasheet skeleton	14
13. The Chipalooza #2 Analog Harness, and What It Means for Your CACE Setup . .	16
13.1 What the harness actually provides	16
13.2 On <code>analog[2:0]</code> vs. a possible <code>analog[3:0]</code>	16
13.3 Consequence for your datasheet’s <code>default_conditions</code>	17
Sources	17

Math note. This document uses $\dots$ for inline math and $\dots$ for display math, which renders on GitHub. GitLab’s default Markdown renderer expects `$`...`$` for inline math instead of $\dots$; if you view this file on GitLab and the inline equations look wrong, that’s why — the display equations ($\dots$) will still render correctly on both.

Two deadlines, both 31 August 2026, two different deliverable types:

Track	Milestone due 31 Aug 2026	What's actually required
SG13CMOS5L	Schematic review	A committed CACE run (docs/<name>_schematic.md) against your xschem schematic — §§1–8 and §13 of this document are your critical path. This is the deliverable this walkthrough is built around.
GF180MCU (“180u”)	Proposal	A written proposal document (same format as your s2d_d2s_pinbuffers.md) — no CACE run required yet. §9.2 (GF180MCU adaptation) is <i>get-ahead</i> reference for after the proposal is accepted, not something you need to finish by the 31st for this track.

Don't let GF180MCU CACE/PDK-porting work compete for time against the CMOS5L schematic-review evidence this week — the GF180MCU track only needs prose and a circuit description right now.

Contents

1. What CACE Is, and Why It Matters for Schematic Review
2. CACE Project Layout
3. Anatomy of the YAML Datasheet
4. Testbench Templates and Token Substitution
5. Running CACE and Reading the Report
6. PVT Corner Sweeps
7. Monte Carlo and Mismatch Analysis
8. Post-Layout Characterization
9. Per-PDK Adaptation: SG13G2/CMOS5L, GF180MCU, SKY130
10. Model-Accuracy Caveats
11. Mapping CACE Output to Chipalooza Schematic-Review Expectations
12. Appendix: Command Cheat-Sheet and Blank Datasheet Skeleton
13. The Chipalooza #2 Analog Harness, and What It Means for Your CACE Setup

1. What CACE Is, and Why It Matters for Schematic Review

CACE (Circuit Automatic Characterization Engine, maintained by efabless) is a Python-based simulation runner for analog and mixed-signal circuits. You write one specification file — the **datasheet**, in YAML — that lists every pin, every operating condition (supply voltage,

temperature, process corner, load, ...), and every electrical parameter you're claiming for the circuit, together with pass/fail limits. CACE then generates the SPICE netlists, runs ngspice for every condition combination, extracts the results, checks them against your limits, and writes a Markdown/HTML report with plots.

Two things make this directly relevant to a Chipalooza schematic-review deadline rather than a "nice to have":

- **The schematic-review artifact is a CACE report.** Efabless's own analog IP convention (see §2) expects a docs/<name>_schematic.md file generated straight from the CACE run against your xschem schematic. A reviewer opening your repo expects to find your claimed specs, the conditions you tested them under, and a pass/fail table — not just a schematic picture and a promise.
- **It forces you to write the spec before/while you simulate,** rather than screenshotting a few waveforms after the fact. As your course material puts it (quoting the *Analog (Integrated) Circuit Design* textbook, §6.5):

"Luckily, there are open-source versions of simulation runners available, and we will use CACE in this lecture. CACE is written in Python and allows one to set up a datasheet in YAML that defines the simulation problem and the performance parameters to evaluate against specified limits." — H. Pretl et al., *Analog (Integrated) Circuit Design*, §"PVT Simulation and Design Corners"

CACE's own documentation (cace.readthedocs.io) frames the same point as a workflow benefit: "CACE encourages good analog design practices, fostering trust in open source analog design." For a multi-team, multi-block shuttle like Chipalooza, that trust is the entire point of the schematic review — the reviewer cannot re-derive your circuit's operating point by eye across 60+ submissions, but they can read a CACE table.

2. CACE Project Layout

Efabless publishes a template repository, `sky130_ef_ip__template`, that fixes the directory convention every Chipalooza-style analog IP block is expected to follow. A real Chipalooza entry — `sky130_sw_ip__por`, a bandgap-referenced Power-on-Reset block — follows it too. Your course's own `cace/` folder (for the 5T-OTA and improved-OTA examples) is the same convention, just embedded inside a textbook chapter instead of standing alone as a repo.

```
<pdk_family>_<handle>_ip__<project>/
├── cace/           ─ CACE datasheet (YAML) + testbench templates
├── docs/          ─ CACE-generated reports:
│                   <name>_schematic.md, _layout.md, _pex.md, _rcx.md
├── xschem/        ─ <name>.sch / <name>.sym (top level), <name>_tb.sch (testbench),
│                   <name>/ (internal subcircuits), xschemrc
├── netlist/       ─ SPICE netlists CACE generates (subfolders: schematic/layout/pex/rcx)
├── mag/           ─ Magic layout (optional; delete if unused)
├── gds/           ─ GDSII
├── ip/            ─ dependency IP blocks, added as git submodules
├── runs/          ─ CACE run output (gitignored)
├── .github/workflows/cace.yml ─ CI: re-runs CACE automatically on push
└── README.md      ─ links to docs/, one line per characterization stage
```

Repository naming convention: `<pdk_family>_<handle>_ip__<project>` (double underscore after ip). The `<handle>` is a short unique identifier — your initials work — to avoid collisions between different designers' blocks of the same name. For this project that would be something like `ihp13g2_cm_ip__s2d_d2s_pinbuffers`.

Two structural points worth internalizing before you build your own tree:

- **docs/ is stage-tagged, not just one report.** CACE can characterize the *same* datasheet against four different netlist sources — schematic, layout (LVS), pex (C-parasitic extracted), rcx (full R-C extracted) — selected with the `-s/--source` flag (see §8). The convention is one report file per stage: `<name>_schematic.md` is what a *schematic* reviewer wants; `_pex.md/_rcx.md` come later once you have layout.
 - **ip/ is how blocks depend on each other**, added as git submodules (`git submodule add ../<ip_block>.git ip/<ip_block>`), with `xschemrc` sourcing each dependency's `xschemrc` recursively. If your gmC filter block reuses an Oguey-Aebischer bias cell as a separate IP, this is the mechanism, not a copy-paste.
-

3. Anatomy of the YAML Datasheet

The datasheet format (current version 5.2, per the CACE reference manual) has six top-level sections. Below, each is explained against the **real, working datasheet from your course** — `cace/voltage-buffer-ota.yaml`, characterizing the 5T-OTA voltage buffer — so this is an excerpt of a file that already runs, not a hypothetical.

3.1 Metadata and authorship

```
name:          ota-5t
description:    Simple voltage buffer for capacitive load realized with 5T-OTA
PDK:           ihp-sgl3g2 # (for IIC-OSIC-Tools version 2025.01 or newer)
cace_format:   5.2

authorship:
  designer:     Harald Pretl
  company:      Johannes Kepler University
  creation_date: August 25, 2024
  license:      Apache 2.0
```

name must match the cell name; PDK is looked up against your installed PDK (no spaces). This is also where you'd write `ihp-sgl3cmos5l` once that PDK identifier stabilizes (see §9).

3.2 Paths

```
paths:
  root:         ..
  schematic:    xschem
  netlist:      cace/netlist
  documentation: cace/_docs
  runs:        _runs
```

root anchors everything else relative to the `cace/` folder's parent — normally your project root. `netlist` is where CACE writes generated SPICE, subdivided automatically by netlist source (`schematic/`, `layout/`, `pex/`, `rcx/`). `documentation` is where the Markdown/HTML report lands — this is your `docs/<name>_schematic.md` target.

3.3 Pins

```
pins:
  vdd:
    description: Positive analog power supply
```

```

    type: power
    direction: inout
    Vmin: 1.45
    Vmax: 1.55
vss:
    description: Analog ground
    type: ground
    direction: inout
ibias_20u:
    description: Bias current input 20uA nom.
    type: signal
    direction: input
vinp:
    description: Voltage positive input
    type: signal
    direction: input
vinn:
    description: Voltage negative input
    type: signal
    direction: input
vout:
    description: Voltage output
    type: signal
    direction: output

```

Pin names here are checked against your schematic/netlist directly — a typo means CACE fails loudly rather than silently simulating garbage. type is one of digital, signal, power, ground; direction is input/output/inout. Vmin/Vmax/Imin/Imax can reference other pins by expression (e.g. $vdd + 0.3$), which is how you'd express, say, an I/O pin's absolute maximum relative to a rail rather than as a hardcoded number.

3.4 Default conditions

```

default_conditions:
  vdd:
    description: Analog power supply voltage
    display: Vdd
    unit: V
    typical: 1.5
  ibias:
    description: Bias current
    display: Ibias
    unit: uA
    typical: 20
  corner:
    description: Process corner
    display: Corner
    typical: tt
  temp:
    description: Ambient temperature
    display: Temperature
    unit: °C
    typical: 27
  cload:

```

```

description: Load capacitance
display: Cload
unit: fF
typical: 50

```

These are the fallback values for every parameter's simulation unless a specific parameter overrides them. The condition **name** (vdd, temp, corner, ...) is meaningful: it must match a `${vdd}`-style substitution token used inside your testbench schematic (see §4).

3.5 Parameters — spec, tool, conditions, plot

This is where most of the file lives. One example, the AC gain/bandwidth parameter block:

```

parameters:
  ac_params:
    spec:
      gain:
        display: Output voltage ratio
        description: Large-signal dc gain
        unit: V/V
        minimum:
          value: 0.97
        typical:
          value: any
        maximum:
          value: 1.03
      bw:
        display: Bandwidth
        description: The -3dB bandwidth of the buffer
        unit: Hz
        minimum:
          value: 10e6
        typical:
          value: any
        maximum:
          value: any
    tool:
      ngspice:
        template: ota-5t-ac.sch
        format: ascii
        suffix: .data
        variables: [gain, bw]
    plot:
      gain_vs_corner:
        type: xyplot
        xaxis: corner
        yaxis: gain
        limits: auto
      bw_vs_corner:
        type: xyplot
        xaxis: corner
        yaxis: bw
        limits: auto
    conditions:
      corner:

```

```

    enumerate: [ss, sf, tt, fs, ff]
vdd:
    minimum: 1.45
    typical: 1.5
    maximum: 1.55
vin:
    minimum: 0.7
    typical: 0.8
    maximum: 0.9
temp:
    minimum: -40
    typical: 27
    maximum: 130

```

Reading this as a reviewer would:

- spec.gain says: *“I claim 0.97–1.03 V/V unity gain, and I will fail my own build if it isn’t.”*
spec.bw says: *“I claim ≥ 10 MHz bandwidth, no claimed upper bound (any).”*
- tool.ngspice.template points at the testbench schematic; variables lists which measured quantities to pull out of the simulation output.
- conditions is the sweep: five process corners $\times$ supply range $\times$ input range $\times$ temperature range, all combined. enumerate gives a discrete list (corners); minimum/typical/maximum on a numeric condition sweeps that variable across its range if a step is also given, or just runs the three named points if not.
- plot says which of those sweeps to actually render as a graph in the report — gain_vs_corner, bw_vs_corner, etc.

minimum/typical/maximum under spec each accept value: any (measure but don’t grade), a concrete value (grade against it, fail if outside), fail: false (measure and report, don’t fail the build on it), and optional calculation/limit overrides for unusual pass/fail logic (e.g. “must be above” vs. “must be below”).

4. Testbench Templates and Token Substitution

Every tool.ngspice.template entry names an xschem schematic file living under cace/templates/. These are ordinary xschem schematics, with one twist: anywhere you’d normally write a fixed SPICE value, you instead write a **CACE{...} token**, which CACE substitutes at run time with the value for that condition in that particular sweep point. From the course’s actual ota-5t-ac.sch testbench (inside a code_shown symbol carrying raw SPICE):

```

.include CACE{DUT_path}
.temp CACE{temp}
.param mc_ok = CACE{sigma=1}
.option SEED=CACE[CACE{seed=12345} + CACE{iterations=0}]

.control
set num_threads=1
op
let dcgain=v(v_out)/v(v_in)
ac dec 101 10 100MEG
meas ac acgain MAX vmag(v_out) FROM=10 TO=100
let f3db = acgain/sqrt(2)
meas ac fbw WHEN vmag(v_out)=f3db FALL=1

```

Three token forms appear here:

- **CACE{name}** — substitute the current value of condition name (e.g. CACE{temp} becomes 27 or -40 or 130 depending on which sweep point is running).
- **CACE{name=default}** — same, but with a fallback if the condition isn't defined for this parameter (CACE{sigma=1} defaults to 1 — Monte Carlo “off” — unless a Monte Carlo parameter block overrides sigma).
- **CACE[expression]** — an arithmetic expression combining tokens, evaluated before substitution (CACE{seed=12345} + CACE{iterations=0} produces a distinct SPICE random seed per Monte Carlo iteration).

DUT_path is a CACE-provided pseudo-condition pointing at whichever generated device-under-test netlist matches the current -s/--source setting — schematic, layout, pex, or rcx (see §8). You never hand-edit that path; it's how the *same* testbench schematic works unmodified whether you're characterizing the ideal schematic or the parasitic-extracted layout.

Practical note for the “software engineer obfuscation” allergy: there is no hidden templating engine here beyond straight string substitution inside a normal SPICE .include/.control block. If you can read a SPICE deck, you can read a CACE testbench — the only new syntax is the CACE{}/CACE[] token itself.

5. Running CACE and Reading the Report

From your project root:

```
cace cace/voltage-buffer-ota.yaml
```

This runs every parameter block in the datasheet, for every condition combination, and writes the annotated results into documentation: (per §3.2) as Markdown, with per-parameter plots as SVG/PNG. Two useful narrowing flags while iterating:

```
cace cace/voltage-buffer-ota.yaml -p ac_params      # only this parameter block
cace cace/voltage-buffer-ota.yaml -j 4             # 4 parallel simulation jobs
```

The Markdown report is the artifact you commit to docs/. If you want an HTML version for easier browsing (not required for submission, just convenient locally), the course ships a one-line wrapper:

```
cace/cace_view.sh cace/_docs/ota-5t_schematic.md
```

— which is nothing more than `pandoc -f markdown -t html` followed by opening the result in a browser. No magic there either.

Reading the report: each parameter gets a pass/fail row (measured value vs. your spec limits) plus, where a plot block exists, a graph swept across whichever condition you asked for (gain_vs_corner, bw_vs_temp, ...). A clean CACE report with everything green is the single strongest piece of evidence you can hand a schematic reviewer — stronger than a paragraph of prose claiming your circuit works.

6. PVT Corner Sweeps

“PVT” = process, voltage, temperature. Doing this by hand — as your course material notes — means either (a) using designer intuition to guess the worst-case corner and simulating only that, or (b) brute-forcing every combination with a runner like CACE. Practically every parameters.*.conditions block you saw in §3.5 is a PVT sweep already:

```
conditions:
  corner:
```



```

    enumerate: [ss, sf, tt, fs, ff]
vdd:
  minimum: 1.45
  typical: 1.5
  maximum: 1.55
temp:
  minimum: -40
  typical: 27
  maximum: 130

```

Five corners $\times$ three supply points $\times$ three temperature points = 45 simulations for one parameter block, run automatically and in parallel (-j). The number of process corners you enumerate is PDK-dependent — ss/sf/tt/fs/ff is the standard 5-corner set (slow-slow, slow-fast, typical-typical, fast-slow, fast-fast) shared across SG13G2, GF180MCU, and SKY130, though the *library file* you point at for each differs (see §9).

A callout from your course material worth repeating here, since it applies directly to a differential buffer/filter block with several independently-biased stages: “*the number of Monte Carlo simulation runs N has to be large enough to approach Gaussian distributions... often $N = 250$ is a good compromise*” — the same balancing act applies to how many corner/PVT points you actually need versus simulation time. Enumerating all 5 corners $\times$ full temperature range for every parameter, every iteration of design, is often more than you need during early sizing; it becomes necessary once you’re locking down the schematic-review numbers.

7. Monte Carlo and Mismatch Analysis

Corners model *global* process variation (an entire wafer skewing slow or fast together). Monte Carlo with device mismatch models *local*, random, device-to-device variation — two nominally-identical transistors on the same die differing slightly — which corners cannot represent and which matters enormously for a differential input buffer’s input-referred offset or a bandgap/V_{cm} reference’s absolute accuracy.

The course’s ota-5t.yaml ac_mc_params block is a complete, working example:

```

ac_mc_params:
  spec:
    gain_mc:
      display: Output voltage ratio (MC)
      unit: V/V
      minimum: { value: any }
      typical: { value: any }
      maximum: { value: any }
    bw_mc:
      display: Bandwidth (MC)
      unit: Hz
      minimum: { value: 10e6 }
      typical: { value: any }
      maximum: { value: any }
  tool:
    ngspice:
      template: ota-5t-ac.sch
      collate: iterations
      format: ascii
      suffix: .data

```

```

variables: [gain_mc, bw_mc]
script: ota-5t-ac.py
script_variables: [gain_mc_arr, bw_mc_arr]
plot:
  gain_mc:
    type: histogram
    xaxis: gain_mc_arr
  bw_mc:
    type: histogram
    xaxis: bw_mc_arr
conditions:
  iterations:           # for Monte Carlo
    description: Iterations to run
    minimum: 1
    maximum: 100
    step: linear
    stepsize: 1
  sigma:
    typical: 1           # 1 = mismatch on; 0 = off (see IHP-Open-PDK issue #149)
  corner:
    typical: tt_stat     # Monte Carlo corner for TT

```

Key mechanics:

- **iterations** is a swept condition from 1 to 100 (your N from §6) — each iteration gets a distinct random seed via the `CACE[CACE{seed=...} + CACE{iterations=...}]` expression shown in §4.
- **collate: iterations** tells CACE to gather all 100 individual results into arrays (`gain_mc_arr`, `bw_mc_arr`) rather than reporting 100 separate rows.
- **script: ota-5t-ac.py** is a small user-supplied post-processing script (this pattern is documented in CACE’s “Using Custom Scripts to Postprocess Simulation Results” tutorial) — here, it just collects and possibly computes statistics (mean/ σ) from the raw per-iteration data before handing it to the histogram plotter.
- **corner: tt_stat** — this is the PDK’s Monte Carlo-enabled variant of the typical corner (naming convention differs per PDK, see §9).

A histogram of `gain_mc_arr/bw_mc_arr` across 100–250 runs is exactly the evidence a reviewer wants for claims like “input-referred offset is within $\pm X$ mV” or “Vcm reference holds to $\pm Y$ mV” — the kind of numbers your proof-of-concept Vcm reference and differential buffers will need. Don’t run Monte Carlo and process corners simultaneously in the same sweep (they answer different questions — see §9 for why combining them is usually meaningless); run corners for global worst-case, Monte Carlo separately for local mismatch/offset statistics.

8. Post-Layout Characterization

CACE’s `-s/--source` CLI flag (full reference in §12) selects which netlist CACE builds `DUT_path` from:

```
-s {schematic,layout,pex,rcx,best}
```

- **schematic** — direct from your xschem schematic capture. This is what a **schematic review** wants, and what you should be running right now.
- **layout** — netlist extracted from layout (via LVS extraction), device-level but without parasitics — mainly a connectivity/consistency check against the schematic.
- **pex** — layout netlist with **C**-parasitic extraction added (coupling/loading capacitances).

- rcx — full **R+C**-parasitic extraction (adds parasitic resistance too — routing IR drop, matching-critical resistor mismatch from real geometry).
- best (default) — use the most complete netlist available, falling back to schematic if no layout exists yet.

The point that matters for your immediate deadline: **the exact same datasheet YAML and testbench templates you write today for the schematic-review submission are what you'll re-run, unmodified, against pex/rcx netlists once layout exists.** You are not writing throwaway schematic-only test infrastructure — you're writing the full characterization harness for the entire project, and the schematic-review milestone is just its first invocation. That's the strongest argument for investing real effort in the datasheet now rather than treating it as a review-day formality.

One thing to plan for architecturally, given your proof-of-concept block mixes fast (gmC oscillator/filter) and slow (bias/Vcm) time constants: parasitic RC from real layout routing will move pole locations more than it moves DC operating points, so bandwidth/phase-margin specs are the ones most likely to visibly shift between schematic and pex/rcx runs — worth flagging in your own spec document (see §13) as a “specification difficulty” rather than discovering it in week 33.

9. Per-PDK Adaptation: SG13G2/CMOS5L, GF180MCU, SKY130

Covered in priority order, matching what's time-critical for you right now.

9.1 IHP SG13G2 (full stack) and SG13CMOS5L (restricted) — primary, immediate

The good news first: SG13CMOS5L is *not* a different device family. Per the IHP-GmbH/ihp-sg13cmos5l repository, it's explicitly the **M1-M4+TM1 metal stack** (4 thin routing layers + 1 thick top metal = “5L”) variant of the same SG13G2 130 nm BiCMOS process your course already uses — same MOSFETs, same .model cards, same device symbols in xschem. The metal-stack reduction is a **back-end-of-line (layout)** change, not a schematic/device-model change. That means:

- Your course's existing cace/voltage-buffer-ota.yaml, testbench templates, and xschemrc setup should port to CMOS5L for **schematic-level** simulation with essentially no changes beyond PDK: ihp-sg13cmos5l (once that identifier is finalized — see caveat below) instead of PDK: ihp-sg13g2.
- The metal-stack restriction only bites once you have layout: fewer routing layers, different parasitic RC extraction, and the digital flow (LibreLane, per the ihp-sg13cmos5l-librelane-template) needs the reduced stack's LEF/tech files. None of that affects your _schematic.md CACE run.

The caveat: the ihp-sg13cmos5l PDK repository's own README currently states it is “*meant to be used only as a temporary storage during the development of the build/compile migration script*” — i.e. this is an actively-moving target as of today (22 Aug 2026), not a frozen, versioned PDK release. Practically:

- **Verify before you rely on it.** Confirm the exact \$PDK/\$PDK_ROOT your Chipalooza toolchain expects, and whether MOSFET statistical/mismatch models (sg13g2_hv_nmos_vfbo_mm, sg13g2_hv_nmos_rsgo, etc. — the same Gaussian mismatch parameters used in your course's ota-5t.yaml ac_mc_params block) are confirmed present and unchanged in the CMOS5L packaging, since these live in the device model files, not the metal stack, but “should be identical” is not the same as “confirmed identical” for a PDK still mid-migration.

- **Fallback plan:** if `ihp-sg13cmos5l` isn't stable enough to simulate against directly this week, run your CACE schematic characterization against plain `ihp-sg13g2` (which you already have working) and note in your submission that the design targets the CMOS5L metal-stack variant of the same process — schematic-level results are expected to be identical since no device models differ. Revisit once the PDK repo stabilizes.

Corner naming: `tt/ss/sf/fs/ff` as in your course examples; Monte Carlo/mismatch corner is `tt_stat` (per the course's own `ac_mc_params` block, referencing IHP-Open-PDK issue #149 for the `sigma/mc_ok` semantics).

9.2 GF180MCU — second priority, time-critical

Not related to CMOS5L by name — a different foundry family (GlobalFoundries 180 nm MCU, also offered in 3LM/4LM/5LM metal-stack variants, which is a separate coincidence of “5-layer” terminology worth not confusing with SG13CMOS5L). If your Chipalooza track (or a second submission) targets GF180MCU:

- Master device model file: `sm141064.ngspice` — BSIM4 (level=54), covers 3.3 V and 6 V MOSFETs, BJTs, diodes, and MIM capacitors.
- **Corners are selected per device family independently** — separate `.lib` statements for MOS/resistor/BJT corners, e.g. typical MOS + slow-slow resistor + fast-fast BJT can be mixed in one testbench, unlike a single unified corner name.
- **Monte Carlo/mismatch:** enabled via `.param sw_stat_mismatch=1` (local device mismatch) and optionally `.param sw_stat_global=1` (adds inter-die global variation on top). A `fet_mc_skew/res_mc_skew/cap_mc_skew` parameter (default 3) controls how many sigma of *global* variation to apply — lowering it tightens the Monte Carlo spread if you have measured data suggesting the default 3σ assumption is pessimistic for your process split. Full syntax: GF180MCU PDK docs, §4.1 Statistical Models.
- Porting your CACE datasheet: `default_conditions.corner` enumeration and the testbench's `.lib` include lines need PDK-specific values; the YAML *structure* (`pins/parameters/spec/conditions`) is unchanged.

9.3 SKY130 — third priority, lower urgency

- Corners selected via `.lib "$PDK_ROOT/sky130A/libs.tech/ngspice/sky130.lib.spice"` `tt` (and `ff/ss/sf/fs`, plus `leak/wafer` variants for specific analyses).
- **Monte Carlo/mismatch** uses a `MC_MM_SWITCH` parameter (`.param mc_mm_switch=1`) wired into `AGAUSS(...)` terms on every BSIM4 card, plus `.option SEED=<value>` for reproducible-vs-varying random draws per run — structurally the same idea as SG13G2's `sigma/mc_ok` and GF180MCU's `sw_stat_mismatch`, different parameter name.
- A useful SKY130-specific shortcut: the pseudo-corner `tt_mm` is “typical corner + local mismatch enabled” in one `.lib` selection, rather than needing a separate switch — check whether an equivalent shortcut corner exists for SG13G2/CMOS5L or GF180MCU before assuming you need the two-step (`corner=tt` + separate mismatch switch) pattern everywhere.
- Explicit warning worth restating for all three PDKs, not just SKY130: **don't run process corners and Monte Carlo mismatch in the same sweep**. Corners already assume every device shifts together in the worst direction; stacking random per-device mismatch on top of an already-worst-case corner represents a vanishingly low-probability combined event, not a realistic one, and will make your spec look artificially bad (or you'll be tempted to loosen limits to compensate, which defeats the purpose). Run them as separate parameters blocks, as the course's own `ac_params` (corners) vs. `ac_mc_params` (Monte Carlo, fixed at `tt_stat`) split demonstrates.

9.4 Summary table

	SG13G2 / SG13CMOS5L	GF180MCU	SKY130
Device models	Same BSIM-type models for both stack variants	sm141064.ngspice, BSIM4 (level 54)	BSIM4 (level 54), per-corner .lib files
Corner selection	Single .lib/corner name (tt/ss/sf/fs/ff)	Independent per device family (MOS/R/BJT)	Single .lib file, corner as argument
Mismatch enable	sigma/mc_ok param (course convention)	sw_stat_mismatch=1 (+ sw_stat_global=1)	mc_mm_switch=1 + AGAUSS()
MC “typical” shortcut	tt_stat corner	— (compose manually)	tt_mm corner
Metal stack caveat	CMOS5L repo is WIP as of Aug 2026 — verify before relying on it	3LM/4LM/5LM variants exist; confirm which your shuttle uses	Single standard stack

10. Model-Accuracy Caveats

Worth stating plainly rather than assuming: **open-source PDK Monte Carlo and mismatch models are, in general, less mature and less silicon-validated than a commercial foundry’s qualified design kit.** They are typically derived from a mix of foundry-supplied statistical corners, published process variation data, and community reverse-engineering/curve-fitting — not decades of proprietary characterization across millions of production die. This isn’t a reason to distrust them wholesale, but it is a reason to treat MC/mismatch *numbers* with more margin than you would in a commercial-PDK tapeout, and to say so explicitly in your submission rather than implicitly overclaiming precision the model can’t actually back up.

Practical mitigations, roughly in order of effort:

1. **Add explicit margin** on any spec that leans on a Monte Carlo/mismatch result (offset, PSRR, absolute Vcm accuracy) rather than taking the simulated 3σ number at face value as your final claimed spec.
2. **Cross-check against published silicon data where it exists** — for SG13G2 in particular, IHP publishes process specification documents (e.g. the SG13G2_os_process_spec.pdf) with measured parameter spreads; sanity-check your Monte Carlo histogram’s σ against those where the parameter overlaps (e.g. threshold voltage mismatch).
3. **Use layout structures matched to known-good references where possible** — your own proposal already leans this direction (“derive the pmos and nmos bias currents from layout structures that match the layout of the current and voltage references on the Chipalooza harness”), which sidesteps absolute-model-accuracy risk by relying on *matching* (a much better-modeled effect than absolute value) rather than absolute device parameters.
4. **Flag known-uncertain parameters explicitly in your submission** rather than silently shipping a tight spec — a reviewer trusts “we simulate ± 5 mV offset with a known $\pm 2\times$ model-confidence caveat, so we’re specifying ± 15 mV” far more than an unqualified “ ± 5 mV” that turns out to be a simulation artifact.

11. Mapping CACE Output to Chipalooza Schematic-Review Expectations

Putting §§1–10 together into what actually goes in front of a reviewer:

- **The schematic** (xschem/<name>.sch, <name>_tb.sch), matching the pin list in your CACE datasheet exactly (CACE will already have caught any mismatch before you got this far, since it checks pin names against the schematic).
- **The datasheet** (cace/<name>.yaml), which is your written specification — a reviewer should be able to read spec: blocks and know exactly what you’re claiming without reading a separate prose document.
- **The generated report** (docs/<name>_schematic.md), run with -s schematic, showing every claimed parameter passing (or explicitly flagged/explained if not) across the PVT and Monte Carlo sweeps from §§6–7.
- **A short human-readable description** of what the block does, its architecture, and any known specification difficulties or open questions (your proposal document is already exactly this format) — CACE doesn’t replace this, it substantiates it.

A reviewer’s real question at this stage is not “does this look like a plausible circuit” (anyone can draw a plausible-looking OTA) but “has this designer actually verified, under a defined and reproducible set of conditions, that the circuit does what they say it does.” A clean, committed CACE run answers that question in a way prose alone cannot.

12. Appendix: Command Cheat-Sheet and Blank Datasheet Skeleton

12.1 Commands

```
# Run everything in the datasheet
cace cace/<name>.yaml

# Run one parameter block only, useful while iterating
cace cace/<name>.yaml -p ac_params

# Explicit netlist source (schematic review => schematic)
cace cace/<name>.yaml -s schematic

# Parallel jobs
cace cace/<name>.yaml -j 4

# Convert schematic to a standalone SPICE netlist (already done internally by cace,
# but useful to inspect what it generated)
# -> see cace/netlist/schematic/<name>.spice after a run

# Convert a CACE Markdown report to HTML for local viewing (course helper script)
cace/cace_view.sh cace/_docs/<name>_schematic.md
```

Full CLI reference: `cace --help`, or cace.readthedocs.io/en/latest/usage/cace_cli.html.

12.2 Blank datasheet skeleton

```
name:          <cell_name>
description:    <one-line description>
PDK:           <pdk_identifier>
cace_format:    5.2
```

```

authorship:
  designer:      <your name>
  email:         <your email>
  creation_date: <date>
  license:       Apache 2.0

paths:
  root:          ..
  schematic:     xschem
  netlist:       cace/netlist
  documentation: cace/_docs
  runs:          _runs

pins:
  <pin_name>:
    description: <text>
    type: power|ground|signal|digital
    direction: input|output|inout
    # Vmin / Vmax / Imin / Imax as needed

default_conditions:
  <condition_name>:
    description: <text>
    display: <short label>
    unit: <unit>
    typical: <value>

parameters:
  <parameter_block_name>:
    spec:
      <result_name>:
        display: <label>
        description: <text>
        unit: <unit>
        minimum: { value: <value>|any }
        typical: { value: <value>|any }
        maximum: { value: <value>|any }
    tool:
      ngspice:
        template: <testbench>.sch
        format: ascii
        suffix: .data
        variables: [<result_name>, ...]
    plot:
      <plot_name>:
        type: xyplot|histogram|semilogx|semilogy|loglog
        xaxis: <condition_or_variable>
        yaxis: <result_name>
        limits: auto
    conditions:
      corner:
        enumerate: [ss, sf, tt, fs, ff]
        # other swept conditions here

```

13. The Chipalooza #2 Analog Harness, and What It Means for Your CACE Setup

Your project folder already contains the harness reference (`sg13cmos5l_chipalooza_harness.pdf`) — the actual infrastructure your `s2d_d2s_pinbuffers` block plugs into. Reading it against your proposal's pin table changes a few “to be discussed” items from open questions into constraints you can design against, and it changes how you should structure the CACE `pins:/default_conditions:` sections.

13.1 What the harness actually provides

- **Per-slot dedicated analog I/O:** two pins, `sN_an[1:0]`, per project slot — matches your proposal's `vin/vout` dedicated-pin concept directly (one slot, two dedicated analog pins).
- **Shared analog bus:** `analog[2:0]` — three shared analog lines, routed to any slot through an analog switch matrix (`ana_route_sel`). This is the harness-level realization of your proposal's `vcm/vdiffp/vdiffn` “shared analog pins” concept — they are not extra pins your block owns, they're a shared resource your block's switch logic taps into.
- **Bias/Vcm generation is centralized, not something your block must synthesize from scratch:** each slot gets `sN_ibias[1]`, `sN_ibias[2]`, and `sN_vbias`, sourced from two shared iDACs and a shared voltage bias generator, configured over SPI (registers `0x15–0x1A`). Your proposal's “to be discussed, 100nA ballpark” `ibias` and “to be discussed, if adjustable” `vbias` are, per the harness, **digitally trimmable from a shared generator** — which changes the design question from “how do I generate a stable 100 nA on-chip” to “what range and resolution do I need the shared iDAC to give me, and how sensitive is my circuit to its trim step size.”
- **Power domains match your table almost exactly:** independently gated `vdd3v3/vss3v3` (analog) and `vdd1v2/vss1v2` (digital, can run at 1.5 V) per slot, separate from the I/O ring's `vddio/vssio` — confirms your proposal's pin table rather than requiring changes.
- **Digital control goes through a shared crossbar, not dedicated pins:** `gpio_di[23:0]` in / `gpio_do[7:0]` out, assigned per-slot via SPI registers (`0x20–0x37` input assignments, `0x40–0x4f` output assignments) rather than your scan chain owning fixed physical pins. Your `ena/reset/outbuf_en/inbuf_en/filter_en/vdiff_en[1:0]/vcmsel[1:0]/scanclk/scancnt[1:0]/scan` signals are all internal-to-your-block-boundary signals that get *bound* to specific `gpio_di/gpio_do` bit positions by harness configuration, not fixed pins your schematic hardwires to package pins.

13.2 On `analog[2:0]` vs. a possible `analog[3:0]`

Per your instruction: treat a 4th shared analog line (`analog[3]`) as *possible* — the harness may be revised before final submission — but don't build a dependency on it existing. Concretely, in your CACE setup:

- **Don't hardcode “3 shared analog pins” as a structural assumption anywhere load-bearing.** Your `pins:` block should describe your own block's boundary (however many `vdiffp/vdiffn/vcm`-equivalent pins *your circuit* exposes) — the harness's bus width is a routing-matrix property external to your block, not a parameter your datasheet needs to know at all. As long as your block's differential I/O count doesn't itself change, `analog[2:0]` becoming `analog[3:0]` is invisible to your CACE datasheet and testbenches.
- **Where it *does* matter** is your standalone/breakout test plan (your proposal's final section) — if you're relying on shared analog lines to route unbuffered internal differential signals out for external measurement (as you describe, for measuring VCM and the unbuffered filter output), a wider shared bus gives you more simultaneous internal test-

node visibility. Write that section so it degrades gracefully with 3 lines (your 9 listed tests already time-multiplex the same 2-3 shared lines across different configurations) rather than assuming a 4th line will be available to run two of those tests simultaneously.

- **Practical hedge:** if you do get a firm answer before final submission, updating from 3 to 4 shared lines is a harness-side/testbench-side change only (which physical analog[n] your test setup taps), not a change to your circuit's own pins: list or its CACE parameter specs.

13.3 Consequence for your datasheet's default_conditions

Because `ibias/vbias` are harness-supplied and digitally trimmed rather than internally generated, model them in your CACE datasheet the same way the course's `ota-5t.yaml` models `ibias_20u` — as an **input pin with a condition sweep**, not as an internal node you're trying to characterize the accuracy of. Your own circuit's job is to be correctly biased *given* whatever the harness iDAC delivers (with its own trim resolution/range as a swept condition, e.g. `ibias: { minimum: 80, typical: 100, maximum: 120, unit: nA }` reflecting expected trim tolerance), not to prove the iDAC itself is accurate — that's the harness team's characterization burden, not yours. This is a meaningful scope reduction for your "specification difficulties" section: it turns "generate a stable low-noise 100 nA reference" from your problem into "operate correctly across the iDAC's specified trim range," which is a much smaller and more tractable CACE sweep.

Sources

- [efabless/cace](#) — CACE tool repository
- CACE Documentation — overview, tutorials, reference manual
- CACE Datasheet Format reference
- CACE Command Line Interface reference
- CACE custom-scripts tutorial
- [efabless/sky130_ef_ip_template](#) — analog IP repository convention
- [efabless/sky130_sw_ip_por](#) — real Chipalooza IP example (bandgap PoR)
- [IHP-GmbH/ihp-sg13cmos5l](#) — SG13CMOS5L PDK repository (WIP status)
- [tatzelbrumm/ihp-sg13cmos5l-librelane-template](#) — LibreLane digital-flow template for SG13CMOS5L
- [IHP-GmbH/IHP-Open-PDK issue #149](#) — `sigma/mc_ok` Monte Carlo semantics
- [IHP-Open-PDK: SG13G2_os_process_spec.pdf](#)
- [GF180MCU PDK docs](#) — Statistical Models Syntax & Usage
- H. Pretl, M. Koefinger, S. Dorrer, *Analog (Integrated) Circuit Design*, Johannes Kepler University (Apache-2.0), and its `cace/voltage-buffer-ota.yaml`, `cace/templates/ota-5t-ac.sch`, `cace/cace_view.sh` — the working CACE example this walkthrough is built around
- [sg13cmos5l_chipalooza_harness.pdf](#) (user-provided) — Chipalooza #2 analog harness specification